

CONTEXTGRAPH: REPAIR MEMORY FOR MISSING REQUIREMENTS

Anonymous authors

Paper under double-blind review

ABSTRACT

Past repairs record requirements that a new issue may leave unstated. We study how retrieving these requirements before editing changes a complete coding agent’s repair. In paired comparisons on three Django development tasks, repairs pass official verification with or without memory, but only the history-informed repairs satisfy the historical requirement together with the current one. The added behavior preserves query parameters, removes redundant uniqueness, or honors an explicit destination timezone. Content interventions show that a behavioral requirement can suffice without its historical implementation. They also expose loss of triggering conditions during use: the same requirement produces different repairs when agents reconstruct different inputs. Further executions identify incorrect applicability and excessive preservation. We organize this evidence in ContextGraph: historical witnesses retain their input constructions and observed operations, and retrieved input relations support checks composed along the current workflow. The resulting method communicates the missing behavior together with the conditions under which it is required.

1 INTRODUCTION

The issue description tells a coding agent what is currently reported as broken. It need not enumerate every behavior a complete repair should preserve. A past repair can supply a requirement absent from the new issue, allowing memory to improve a repair that already passes the target’s tests. The target’s success score can then remain unchanged while the repair gains a required behavior. This motivates measuring memory’s contribution at the level of the requirement it supplies.

Consider an administrative redirect that must preserve a deployment prefix. Changing its destination from `request.path_info` to `request.path` fixes that reported fault. However, appending a slash to the resulting path still discards query parameters. A different repair of the same API records this omitted behavior and replaces path concatenation with `request.get_full_path(force_append_slash=True)`. The useful information in this experience is the requirement to retain the query, which must hold together with the deployment-prefix requirement.

We investigate three questions: what a historical repair adds to the current issue, what information suffices to communicate that addition, and how its conditions can be lost during use. The unit of analysis is a current and historical requirement exercised together on an actual repair. This order determines the memory architecture. We first identify information that changes a repair’s behavior, then represent the relations needed to construct that information, and finally examine how candidate selection can retain it. A retrieved rule becomes useful when its requirement survives adaptation to the current operation and enters the agent’s implementation.

2 CONTEXTGRAPH: MEMORY AS EXECUTABLE REQUIREMENTS

Memory’s usefulness depends on the current repair. A historical condition can supply a missing requirement, already hold on the current workflow, or conflict with a change the issue requires. Our comparisons distinguish these cases by executing the current and historical requirements together on repairs produced with and without memory.

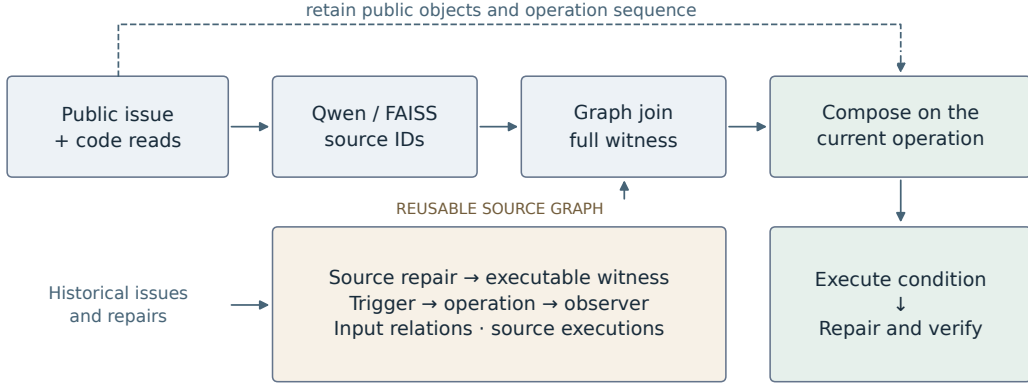


Figure 1: ContextGraph separates candidate selection from condition construction. FAISS returns source identities; the graph returns their complete witnesses and input/operation relations. Supported composition retains the public workflow while introducing a historical input condition. Execution makes its result available during repair. Source evidence is reusable; the composed check and its result belong to the current task.

2.1 STORE A REQUIREMENT WITH THE INPUT THAT SUPPORTS IT

For a historical repair, let V^- and V^+ denote the original and repaired versions. We represent its memory as

$$m = (r, W, E^-, E^+), \quad W = (I, A, O), \quad (1)$$

where r states a behavioral requirement, I constructs an input, A performs the relevant operation, and O observes whether its outcome meets r . E^- and E^+ record executions of this same witness on the two source versions. This separation keeps an exception in the checked operation available to the observer while distinguishing it from failure to prepare the fixture. It also allows the agent to reuse the requirement without copying an obsolete implementation.

A source writer receives the historical issue, patch and relevant code to construct the requirement and witness. Normal executions changing from false on V^- to true on V^+ identify an input that exhibits the source repair. Inputs passing on both versions can provide neighboring preservation conditions. They answer a different question: which behavior remains valid around the trigger? A passing ordinary tuple and a passing tuple subclass, for example, constrain different constructor rules. The input’s relation to the operation therefore belongs in memory alongside its concrete values.

2.2 USE GRAPH RELATIONS TO CONSTRUCT THE RETRIEVED CONDITION

The graph links a source repair to its requirement, witness and before/after executions. Executions connect the witness to the functions it actually reaches. An input relation connects a constructor to the argument supplied to an operation and to the values observed in that source execution. These edges serve distinct roles: the source edge identifies the supporting experience, the operation edge supplies a possible connection to current code, and the input edge tells a composition operator which construction to carry forward.

For supported standalone range programs, assignment and branch analysis follows named-tuple factories to the lookup argument they supply. Consider a program that defines multiple factories but passes only one to `filter(value__range=bounds)`. Selecting any named tuple mentioned in the program can change its fields or calling convention. Following the binding of `bounds` instead returns the constructor used by the executed lookup. An overwritten or unresolved binding supplies no such input relation; the original witness remains available.

At retrieval time, a reader inspects the public issue and original repository to identify current functions. Qwen/FAISS similarity selects distinct historical sources from the same repository, and a graph join returns each source’s requirement, complete program, execution observations and available relations. The current implementation retains the top three witnessed sources in similarity order.

Operation connections accompany them rather than acting as a hard filter: early localization of the reported fault can omit another operation whose behavior the repair must preserve. Figure 1 makes this division explicit. Similarity proposes a source; the graph supplies the information needed to examine its condition in the current workflow.

2.3 COMPOSE INPUTS WHILE RETAINING THE CURRENT OPERATION

When an issue provides an executable example, a composition operator can retain its object construction and operation sequence while changing one input shape. Our Django ORM adapter lifts a scalar lookup `field=x` to `field__range=R(x, x)`, where R is the two-endpoint named tuple from the historical witness. It retains the public construction of x and its annotated query; an ordinary tuple point range supplies a neighboring input. The historical repair establishes the source condition, while the new combination may require a further repair. This separates two reusable parts of memory: the historical input relation and the current operation through which that relation must remain valid.

Writing the public example as $P_q(x)$, the supported transformation is

$$P_q(x) \mapsto P_q^{\text{range}}(R(x, x)). \quad (2)$$

The superscript changes the lookup to a range; the public object’s construction and surrounding query remain in place. For a lazy User inside an annotated subquery, replacing the object with a lazy integer would remove the interaction that makes the combined condition informative. Likewise, replaying the historical named-tuple query alone would omit the current subquery path. The point range preserves the scalar endpoint while adding the historical constructor constraint. The ordinary-tuple point range helps distinguish a failure shared by iterable endpoints from one specific to named tuples.

We implement this transformation over the public program’s syntax tree and the retrieved source input relation. It is a finite adapter for this operation; other operations require a transformation that preserves their own semantics. The resulting check retains an explicit observer. Query acceptance on an empty fixture tests whether preparation succeeds, while a separate nonempty fixture tests which rows are selected. These observations express different requirements and can guide different parts of the repair.

2.4 DELIVER CONDITIONS AND USE THEIR EXECUTION RESULTS

A complete SWE-agent receives the target issue and retrieved memory before browsing the repository, editing code, running tools, and submitting a patch. Historical records contain source-partition issues and their reference patches. Frozen BM25 rankings or cosine similarity with Qwen3-Embedding-8B select the sources (Zhang et al., 2025). We supply the stored rule, original issue and patch, or source-generated behavioral and implementation components. The complete-agent comparisons use DeepSeek V4 Pro0423 through OpenRouter, temperature 0, with budgets matched within each comparison.

Measure the requirements together. We immediately evaluate every submitted repair with the official SWE-bench verifier (Jimenez et al., 2024). We execute the source witness unchanged on the patched target. An issue check measures the reported requirement, and a joint input exercises it together with the historical requirement. The joint input is essential when the requirements interact. Passing the current scalar query and the historical named-tuple query separately does not imply that a named-tuple bound works in the current nested query. Conversely, failure of an isolated historical input does not establish a defect in a workflow that transforms or removes that input before the checked operation.

Return a failed condition for refinement. When a source-validated check executes normally and fails on a candidate, the memory interface can return its unchanged program and actual failed observations for a further repair. The callable interface also executes supplied conditions on the agent’s current working tree, retaining their original inputs and observers across edits. The failure must be interpreted in the current operation: an isolated historical example can fail while the workflow being repaired already satisfies that condition. The observer judges failures of the checked operation

Table 1: Joint current/historical behavior in complete-agent comparisons. Each displayed repair passes official verification. The columns compare the actual repairs with and without memory; the rows retain their distinct task requirements.

Target	Historical addition	No memory	Memory
Django14404	Preserve query string	Fail	Pass
Django12708	Delete redundant uniqueness	Fail	Pass
Django11138	Honor destination timezone	Fail	Pass

against the stated requirement. Passing conditions produce no failure feedback; failures in fixture preparation or observation remain unassessed. We compare this revision with reviewing the same requirement again.

This interface gives three observable stages of use: the condition is delivered, the agent executes it, and the repair incorporates the behavior it specifies. We inspect the actual trajectory to distinguish them. A tool call can reveal a failure that the final patch leaves unchanged; a textual requirement can also lead the agent to construct its own successful check. The method therefore evaluates the submitted implementation, including joint and neighboring inputs, instead of treating retrieval or execution alone as successful memory use.

3 HISTORY SUPPLIES A MISSING REPAIR REQUIREMENT

We use a SWE-bench Verified split with 280 source tasks, 70 development tasks, and a separate 150-task evaluation partition unused by these experiments. Table 1 shows three development comparisons in which memory changes whether the repair meets a requirement absent from the current issue. Django14404 receives the first-ranked source issue and patch. Django12708 and Django11138 come from a fixed six-task cohort selected by high Qwen similarity; its memory arm receives three unchanged Flat rules per task. Both arms resolve all six cohort tasks under official verification. The two displayed cases distinguish additional historical behaviors, rather than increasing that official score.

An omitted behavior of the same API. Django14404 reports that administrative redirects lose a deployment prefix. Both agents fix the prefix and pass official verification, but only the repair informed by source16612 also preserves the query string. The ordinary agent’s actual observations include the implementation that retains `QUERY_STRING`; seeing an available implementation does not by itself make query preservation part of its repair. A joint input combines the prefix with repeated and percent-encoded query values, `keep=a%2Fb&keep=c%20d`. Only the history-informed repair preserves both.

A neighboring operation. Django12708 reports ambiguous deletion of an ordinary index when a unique constraint uses the same columns. The first retrieved source, 15732, instead concerns deletion of a unique constraint sharing its column with a primary key. Both repairs fix ordinary-index selection; only the memory-informed repair also excludes primary keys when deleting unique constraints. The source-generated PostgreSQL check runs unchanged on both target repairs. A separate combined input creates an ordinary index, redundant uniqueness, and a primary key on the same column. Ordinary-index deletion already succeeds on this target-base input; both repairs retain it, but only the memory-informed repair also deletes redundant uniqueness while retaining the primary key and data. History supplies a requirement for the neighboring operation.

An independent argument of the same operation. Django11138 requires date conversions to respect the database’s source timezone. Source13363 requires `TruncDate` and `TruncTime` to honor an expression’s explicit destination timezone. Both agents repair the source timezone, but only the memory-informed repair also honors the destination. A source-generated check distinguishes the historical versions and runs unchanged on the target repairs. On an independent SQLite input combining a UTC+1 database with an explicit New York destination, only the memory-informed repair returns the correct date and time. The issue and history constrain different ends of the conversion.

Table 2: One fresh complete-agent attempt per input and known development task. All eight pass official verification. The two joint columns measure the original current/historical combinations. The MySQL SQL-generation and named-zone ORM cases are additional conditions specified after inspecting completed timezone repairs and then executed on every timezone submission.

Memory input	Constraint joint	Timezone joint	MySQL cases	Named-zone cases
None	Fail	Fail	Pass	Fail
Requirement	Fail	Pass	Pass	Pass
Implementation	Pass	Pass	Pass	Fail
Both	Pass	Pass	Fail	Pass

The addition can guide further repair. Two fresh agents start from the same completed ordinary timezone patch. Issue-based additional review returns it unchanged; supplying a generated historical check and its failed observations leads the other agent to use `self.get_tzname()` in both truncation methods. Both revisions pass official verification, but only the latter passes the historical and joint inputs. The intervention supplies corrective information that further issue-based review does not recover in this comparison.

4 WHAT INFORMATION MAKES MEMORY USABLE?

A requirement can suffice without the source implementation. On the redirect task, replacing the first entry of the existing Flat packet with its original issue and patch yields a byte-identical repair; both forms preserve the joint behavior. A separate comparison starts two new agents from the completed ordinary patch. Both receive the source-generated sentence, “The redirect location from AdminSite.catch_all_view includes the original query string.” One also receives the generated check and its failed observations. Both submit byte-identical repairs that pass official and joint checks. The sentence-only agent finds `get_full_path()` in the current repository: memory communicates the desired behavior while the agent locates its implementation. Executable feedback adds no observed behavior in this pair.

Which information makes the requirement usable? We select the constraint and timezone benefit cases and run four fresh complete agents per task, fixing the single historical source and its delivery before editing. Two source-only constructions produce a behavioral requirement and an implementation reference; the combined input concatenates them unchanged. All eight repairs pass official verification (Table 2). On Django12708, only Implementation and Both also remove redundant uniqueness while preserving the primary key. The Requirement agent tests a field with `unique=True`, but places `unique_together` on a different, two-column set. This nearby passing test loses the same-column condition stated in memory. On Django11138, all three memory inputs preserve the explicit destination timezone, while the fresh no-memory repair omits it. The Requirement agent finds `self.get_tzname()` in the target implementation without being given that helper in memory.

The timezone repairs also differ on further conditions. Both still generates an unnecessary MySQL conversion between identical zones; Implementation instead misinterprets a named Paris source timezone, producing New York time 19:21 instead of 17:30 on a summer input. Requirement passes both additional diagnostics. These concrete omissions distinguish carrying a requirement, applying its triggering condition, and implementing the resulting behavior correctly. Two subsequent original-base agents receive the unchanged constraint Requirement, with or without a source-validated example; both pass official and joint checks. Thus the earlier failure does not establish that an implementation reference is necessary: the same requirement can yield different repairs depending on how its trigger is reconstructed.

Execution feedback makes an omitted condition concrete. Two fresh agents refine the same original failed Requirement candidate, both receiving its requirement again. One also receives the unchanged source-generated check and its actual failures for a field with a primary key or `unique=True`. Requirement-only review excludes the primary key but still fails the ordinary unique-field condition. Feedback adds generated-name disambiguation and passes both conditions.

	Official verification	Historical named tuples (2)	Named tuple + lazy User (1)	ADDITIONAL DIAGNOSTIC Ordinary tuple/list subclasses (2)
Memory supplied for revision				
Original issue + patch	Pass	Pass	Pass	Pass
Requirement sentence	Pass	Fail	Fail	Pass
Requirement + check + execution failures	Pass	Pass	Pass	Fail

Figure 2: Memory content changes which behaviors a repair preserves. One revision per input starts from the same Django12663 candidate and source13590. Parentheses give case counts; each cell reports all cases in its group. The ordinary-subclass inputs were chosen after inspecting the feedback repair. All three repairs pass official verification, while only original history passes every displayed behavior.

Both revisions pass official verification and the withheld joint sequence. The added behavior is recovery of the ordinary unique-field condition identified by feedback.

5 APPLICABILITY IS PART OF REPAIR MEMORY

Recovering an example can overgeneralize its rule. On Django12663, both a three-source packet selected through code connections and a ten-source similarity packet contain source13590, yet neither repair recovers its named-tuple range behavior. Three subsequent fresh revisions hold that source and the initial candidate fixed. Figure 2 separates official success from recovery of the historical requirement and preservation of related inputs. Original history and execution feedback recover the named-tuple behavior, including its withheld combination with a lazy User; the requirement sentence alone does not. A subsequent review with the same budget and an empty reference returns the initial candidate unchanged and still fails these behaviors. The feedback repair, however, expands every non-builtin sequence into positional constructor arguments. This recovers named tuples but breaks ordinary tuple and list subclasses, which still work on the original-history repair, shared initial candidate and unmodified target. The missing distinction is the constructor’s calling convention: accepting a named tuple does not justify expanding every sequence subclass. Useful repair memory must retain the condition under which its example supports the implementation rule.

Execution can stop short of adoption. In a selected SQLite refinement pair, both agents receive the same candidate fixing qualified columns in CHECK expressions, together with a source-generated table-name requirement and its executable examples. One also receives the examples’ actual failures on that candidate. Both agents reproduce the reserved-name table failure, but only the feedback agent changes identifier quoting; historical review returns the candidate unchanged. Both pass official verification, while only the feedback repair also completes CHECK migration followed by an explicit named-table integrity check on a reserved-name table.

Construct a boundary that separates the repairs. A source-only writer constructs a named-tuple trigger and an ordinary-tuple neighbor using the same executed query and row observer. The trigger distinguishes the historical versions, but the neighbor passes on both source versions and all five measured target versions, including the overgeneralized feedback repair. Replacing that neighbor with a subclass inheriting its container’s behavior, while keeping the bounds unchanged, passes on both source versions and fails only on the feedback repair among the five targets. Varying the input’s type relation exposes a preservation condition that ordinary values miss.

Compose requirements beyond the corrected source examples. A fixed timezone adapter extracts three zone values from the historical witness, varies their database, ambient and destination roles, and adds a 182-day input shift. Across 72 combinations, the historical repair improves 20, preserves 20, and leaves 32 incorrect; every operation executes normally. We apply the unchanged

Table 3: Added conditions in the original-base Django12663 pair. Both inputs contain the same history, current program and failure, tool, and execution instruction. Only the second adds the composed programs and their failures. Each row counts the named checks, not independent tasks.

Measured behavior	Current feedback	+ Composed conditions
Official target verification	Pass	Pass
Historical named-tuple examples	0/2	2/2
Withheld nonempty joint queries	2/3	3/3
Withheld ordinary subclasses	2/2	2/2
Mechanical scalar / named tuple / tuple	2/3	3/3

family to all four earlier timezone content-comparison repairs. Requirement, Implementation and Both each pass the 40 combinations supported by the repaired source. Yet Implementation fails all 32 remaining combinations, including 16 that the no-memory repair passes; Requirement and Both pass all 72. This known-case construction exposes a limit of selecting conditions solely by success on the repaired source. Completing the current repair requires combining source-timezone interpretation with the historical destination requirement, including configurations that neither historical version handles correctly.

Reconstruct the missing combination. In a further construction study on the known range-query case, a writer receives the public issue and historical source witness, with candidate repairs withheld. After a public-example correction from lazy integers to lazy Users, its program combines lazy Users with named-tuple bounds in one nested subquery; the historical case distinguishes the source versions. On the earlier partial history repair, its current and historical cases each pass while its joint case fails. The constructed condition detects a gap that replaying the historical example alone would miss. Starting four fresh refinements from that partial repair, the historical packet alone yields one of two repairs passing all seven measured conditions; appending the constructed program and its failing observation yields two of two. All four pass official verification. Both augmented attempts execute the joint input and repair its second container reconstruction; the unsuccessful historical attempt checks ordinary named-tuple ranges and submits the initial patch unchanged. The deterministic point-range adapter reaches the same omission while retaining the original public query’s construction: on that partial repair, the scalar and ordinary-tuple queries are accepted, but the named-tuple query raises a missing-endpoint constructor error. This construction checks query acceptance on an empty fixture, preserving the lazy User and nested annotation that a regenerated example had replaced. Delivery alone does not ensure use: in a separate original-base pair, both repairs fail official and nested-query verification. The augmented agent recovers the standalone history but never executes the joint input, accepting the continued User type error in its own check. Both input and observer must survive use to guide repair.

Fix the current feedback to test the added condition. A new original-base pair receives the same history, public scalar program, its actual failure, a callable execution interface, and instructions to run supplied conditions before editing and submission. Only the augmented input adds the composed named-tuple and ordinary-tuple programs with their original-code failures. Both repairs pass official verification (Table 3). The current-only repair fails both historical examples and the withheld named-tuple/lazy-User query; the augmented repair passes these and the other four withheld joint and subclass checks. The augmented agent executes the joint input before editing, then uses it again after removing a global field conversion that broke an existing test. The input exposes a remaining omission in range-endpoint preparation, which the agent repairs. In this pair, composition makes the historical condition concrete along the current operation while the current feedback is held fixed.

Initial localization can omit behavior the repair must preserve. On this same known case, requiring a recorded operation connection from a pre-repair public-code localization excludes source13590, which FAISS ranks first. Retaining the similarity candidate lets its graph input relation construct the exact joint program used above. This result motivates retaining similarity candidates while using graph relations to construct their conditions along the current operation.

Relevant history need not add a missing requirement. In a fixed four-repository development cohort, sixteen original-base attempts compare no memory, Requirement, Implementation, and Both using the same top-ranked source per task. The first two inputs each resolve four of four tasks; the latter two each resolve three, omitting Xarray4687’s public `keep_attrs` argument. The unmodified Django15499 and Xarray4687 targets already pass their selected source examples for operation reduction and independent attribute mappings. These memories concern behavior already satisfied on the observed inputs.

The current operation changes the historical condition’s role. Matplotlib20826 reports lost tick visibility after clearing shared axes. The second-ranked historical source concerns finite data bounds after plotting on a twin axis. Its two source-validated examples fail unchanged on the target and enter the current clearing function. Yet a paired execution with the same shared plots shows that clearing and replotting restores finite bounds, while losing tick visibility; retaining the plots preserves visibility but loses finite bounds after twin plotting. A complete no-memory repair then restores tick visibility and preserves finite bounds in the clear/replot workflow, passing official verification. The isolated historical examples still fail. Thus a historical failure and a shared call path do not identify what this repair lacks: the operation sequence determines whether the condition needs recovery or preservation.

Preservation must allow the required change. A further three-task development comparison holds the retrieved sources fixed and supplies historical requirements or concrete edit advice. Both Django16819 repairs cancel a same-name `RemoveIndex/AddIndex` sequence, although changing its indexed column must take effect; a subsequent SQLite diagnostic exposes the lost replacement. Both Xarray7229 repairs instead preserve coordinate attributes in scalar `where` cases that require dropping them, failing official target verification. The useful boundary must specify both what remains valid and when the current operation requires change.

Historical memory can already overstate that boundary. A source-only Xarray4094 contract claims a stack/unstack roundtrip preserves sample coordinates when variables have only sample dimensions. The source patch repairs the reported three-element dataset, but a one-element input satisfying the same condition loses its sample dimension and coordinate. This package is rejected before target use: explicit applicability fields do not establish the repaired source’s supported scope.

6 RELATED WORK

MemOp trains memory writing from measured gains in task success, localization and efficiency (Guo et al., 2026); ExpGraph learns graph retrieval from task scores with and without a retrieved experience packet (Feng et al., 2026). Our interventions identify source-specific behaviors that can differ while the official target score stays unchanged. ExpeL distills reusable insights from experience (Zhao et al., 2024). ExpeRepair uses historical test demonstrations and reflective insights to generate reproduction and boundary tests (Mu et al., 2026). RETester transforms historical bug templates while retaining the applicability of the original refactoring operation (Wang et al., 2025). MIRA separates SQL corrections into reusable requirements and uses database checks to guide their activation and adaptation (Liu et al., 2026). BUGSTONE-E2E extracts scan anchors and vulnerability conditions from fixing commits, then applies semantic and runtime checks to matching target sites (Wu et al., 2026). TRACE compiles user corrections into atomic rules and runtime checks for later coding-agent tasks (Zhou et al., 2026). TDAD selects checks through code–test dependencies (Alonso et al., 2026); Agent-CoEvo evolves code and tests within an issue (Li et al., 2026); coupled synthesis combines dependency graphs and execution feedback (Villuri et al., 2026). Our interventions isolate how historical conditions survive adaptation to the current operation and enter its repair.

AI USE STATEMENT

Generative AI assisted literature review, method development, implementation, experiments, analysis, and writing.

REFERENCES

- Pepe Alonso, Sergio Yovine, and Victor A. Braberman. TDAD: Test-driven agentic development – reducing code regressions in AI coding agents via graph-based impact analysis, 2026. URL <https://arxiv.org/abs/2603.17973v2>.
- Tao Feng, Chongrui Ye, Tianyang Luo, Jingjun Xu, Xueqiang Xu, Haozhen Zhang, Zhigang Hua, Yan Xie, Shuang Yang, Ge Liu, and Jiaxuan You. ExpGraph: Model-agnostic experience learning with graph-structured memory for LLM agents, 2026. URL <https://arxiv.org/abs/2605.30712v1>.
- Xuehang Guo, Zora Zhiruo Wang, Qingyun Wang, Graham Neubig, and Xingyao Wang. Enhancing software engineering through closed-loop memory optimization, 2026. URL <https://arxiv.org/abs/2606.05646v1>.
- Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. Swe-bench: Can language models resolve real-world github issues?, 2024. URL <https://arxiv.org/abs/2310.06770>.
- Kefan Li, Yuan Yuan, Mengfei Wang, Shihao Zheng, Wei Wang, Ping Yang, Mu Li, and Weifeng Lv. Beyond fixed tests: Repository-level issue resolution as coevolution of code and behavioral constraints, 2026. URL <https://arxiv.org/abs/2604.04580v1>.
- Yining Liu, Chenyu Yang, Boyan Li, Rui Mao, and Yuyu Luo. MIRA: Evidence-verified repair memory for text-to-sql correction, 2026. URL <https://arxiv.org/abs/2608.06950v1>.
- Fangwen Mu, Junjie Wang, Lin Shi, Song Wang, Shoubin Li, and Qing Wang. ExpeRepair: Dual-memory enhanced LLM-based repository-level program repair, 2026. URL <https://arxiv.org/abs/2506.10484v2>. Version 2; accepted at FSE 2026.
- Gnaneswar Villuri, Hashmath Shaik, and Alex Doboli. Dynamic adaptation of the llm context for generating routines with coupled semantics, 2026. URL <https://arxiv.org/abs/2609.04570v1>.
- Haibo Wang, Zhuolin Xu, and Shin Hwei Tan. Testing refactoring engine via historical bug report driven llm, 2025. URL <https://arxiv.org/abs/2501.09879v2>.
- Qiushi Wu, Kevin Eykholt, Youngja Park, Xiaokui Shu, Dhilung Kirat, Douglas Lee Schales, and Ian Molloy. The history is the detector: Executing CVE patch history, end-to-end, 2026. URL <https://arxiv.org/abs/2609.05335v1>.
- Yanzhao Zhang, Mingxin Li, Dingkun Long, Xin Zhang, Huan Lin, Baosong Yang, Pengjun Xie, An Yang, Dayiheng Liu, Junyang Lin, Fei Huang, and Jingren Zhou. Qwen3 embedding: Advancing text embedding and reranking through foundation models, 2025. URL <https://arxiv.org/abs/2506.05176v3>.
- Andrew Zhao, Daniel Huang, Quentin Xu, Matthieu Lin, Yong-Jin Liu, and Gao Huang. Expel: Llm agents are experiential learners, 2024. URL <https://arxiv.org/abs/2308.10144v3>.
- Yujun Zhou, Kehan Guo, Haomin Zhuang, Xiangqi Wang, Yue Huang, Zhenwen Liang, Pin-Yu Chen, Tian Gao, Nuno Moniz, Nitesh V. Chawla, and Xiangliang Zhang. Getting better at working with you: Compiling user corrections into runtime enforcement for coding agents, 2026. URL <https://arxiv.org/abs/2606.13174v1>.